# UBUNTU VM PRACTICALS

## Beginner’s Step-by-Step Guide

7 practicals • Ubuntu • VMware Workstation / VirtualBox • Apache • HAProxy • Python

*A command-by-command guide explaining which VM to use, what to type, how to test, and what results to expect.*

# How to Use This Guide

Follow the practicals in order. Every command block should be run in the Ubuntu terminal on the VM named in the heading immediately above it. Do not run Ubuntu commands in Windows Command Prompt.

## What you need

- A Windows/Linux computer with VMware Workstation or VirtualBox installed.
- Ubuntu Server or Ubuntu Desktop ISO. Internet access is needed inside the VMs to install packages.
- Enough memory to run two or three VMs. For a light lab, start with 1 CPU and 1–2 GB RAM per VM, if your computer can support it.
- Three VMs for load-balancer practicals: VM1 = Web Server 1, VM2 = Web Server 2, VM3 = Load Balancer. Some practicals need only VM1 and VM2.

## Create and connect the VMs

1. Create or install Ubuntu in the VM software. Complete the Ubuntu setup and remember the username and password.
2. Set the network adapter of every VM to the same private network. In VMware, NAT is usually simplest. In VirtualBox, use a shared NAT Network (not separate default NAT adapters), or use a host-only network plus a NAT adapter for internet access.
3. Start all VMs. Open a terminal in each VM. On Ubuntu Desktop, press Ctrl+Alt+T. Ubuntu Server normally opens directly to a terminal.
4. Optional but helpful: set distinct hostnames. Run the matching command on each VM.

On VM1:

```
sudo hostnamectl set-hostname web1
```

On VM2:

```
sudo hostnamectl set-hostname web2
```

On VM3:

```
sudo hostnamectl set-hostname lb1
```

If you changed the hostname, log out and back in or restart the VM so the new name is reflected everywhere.

## Find and record private IP addresses

Run this separately on each VM:

```
hostname -I
ip -br addr
```

Write down the private-network IP address. Use the same IP in all later commands. Do not use 127.0.0.1; that address refers to the current VM itself.

| VM | Role | Private IP (fill in your actual value) |
|---|---|---|
| **VM1** | Backend web server 1 | _____ |
| **VM2** | Backend web server 2 | _____ |
| **VM3** | Load balancer | _____ |

## Check connectivity before configuring services

From VM1, test VM2. From VM3, test both backend VMs. Replace the placeholder with the IP you recorded:

```
ping -c 4 VM2_IP
ping -c 4 VM1_IP
```

A successful test shows replies with time values. If it fails, confirm all VMs use the same private network, check the IPs, and check firewall settings.

## How to paste and save commands

- Type commands exactly as shown. Replace uppercase placeholders such as VM1_IP with real values; do not type the angle brackets or leave the placeholder unchanged.
- When a command asks for your password after sudo, type the Ubuntu password. The terminal does not display the password characters; press Enter when finished.
- When using nano: Ctrl+O saves, Enter confirms the filename, and Ctrl+X exits.
- If a command is still running in its own terminal, such as a Python web server, leave that terminal open. Use another terminal for the next commands. Ctrl+C stops the running server.

**Important:** Keep this lab on a private virtual network. Do not expose lab ports to the public internet. Do not put real student details, passwords, fee records, or exam submissions into these demo pages.

# Quick Reference: HAProxy Configuration

Practicals 1, 2, 4, and 6 use the same three-VM topology. Their backend pages and backend names differ, but the configuration procedure is the same.

## Install packages on the VMs

On each backend VM (VM1 and VM2):

```
sudo apt update
sudo apt install apache2 curl -y
sudo systemctl enable --now apache2
sudo systemctl status apache2
```

On the load balancer VM (VM3):

```
sudo apt update
sudo apt install haproxy curl -y
sudo systemctl enable haproxy
```

## Replace the HAProxy configuration safely

Back up the default file first. The examples below use a complete config so the default port-80 listener does not conflict with the new listener.

```
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.backup
sudo nano /etc/haproxy/haproxy.cfg
```

In nano, remove the existing file content and paste the complete configuration provided in the practical. Save with Ctrl+O, Enter, then exit with Ctrl+X. Do not leave two frontends both binding to port 80.

## Useful checks

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl restart haproxy
sudo systemctl status haproxy
sudo journalctl -u haproxy --no-pager -n 30
```

The configuration check should report that the configuration is valid. If it reports an error, review the line mentioned and ensure that all IP addresses and indentation are correct.

## Firewall note

Check the status before changing firewall rules:

```
sudo ufw status
```

If UFW says “inactive”, you do not need to add a UFW rule. If UFW is active, allow only the needed private-network traffic. For an HTTP backend on port 80, run this on each backend, replacing LOAD_BALANCER_IP:

```
sudo ufw allow from LOAD_BALANCER_IP to any port 80 proto tcp
```

If UFW is active on VM3, allow the client-facing HTTP port:

```
sudo ufw allow 80/tcp
```

**Important:** Do not enable UFW for the first time during this lab unless you know the required SSH and network rules. A new firewall rule can interrupt remote access.

# Practical 1 — Load Balancing and Fault Tolerance

Goal: VM1 and VM2 each run an Apache web server with a different response. VM3 uses HAProxy to distribute requests. If one backend stops, HAProxy should route requests to the backend that remains healthy.

## Step 1: VM1 — install Apache and create its page

```
sudo apt update
sudo apt install apache2 curl net-tools -y
sudo systemctl enable --now apache2
hostname -I
sudo tee /var/www/html/index.html > /dev/null <<'EOF'
<h1>Response from Web Server 1 (VM1)</h1>
EOF
curl http://localhost
```

## Step 2: VM2 — install Apache and create its page

```
sudo apt update
sudo apt install apache2 curl net-tools -y
sudo systemctl enable --now apache2
hostname -I
sudo tee /var/www/html/index.html > /dev/null <<'EOF'
<h1>Response from Web Server 2 (VM2)</h1>
EOF
curl http://localhost
```

Copy each VM IP shown by hostname -I. From VM3, test direct connectivity to both backends before configuring HAProxy:

```
curl http://VM1_IP
curl http://VM2_IP
```

## Step 3: VM3 — configure HAProxy

```
sudo apt update
sudo apt install haproxy curl -y
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.backup
sudo nano /etc/haproxy/haproxy.cfg
```

Replace the entire file with the following. Replace the sample IP addresses with the real VM1 and VM2 private IP addresses:

```
global
    log /dev/log local0
    log /dev/log local1 notice
    daemon
    maxconn 2048

defaults
    log global
    mode http
    option httplog
    timeout connect 5s
    timeout client 30s
    timeout server 30s

frontend http_front
    bind *:80
    default_backend http_back

backend http_back
    balance roundrobin
    option httpchk GET /
    server vm1 192.168.1.101:80 check
    server vm2 192.168.1.102:80 check
```

**Important:** In the config above, 192.168.1.101 and 192.168.1.102 are examples only. Replace them with your actual addresses before saving.

Validate the configuration, start HAProxy, and record VM3 IP:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl enable --now haproxy
sudo systemctl restart haproxy
sudo systemctl status haproxy
hostname -I
```

## Step 4: test load balancing

```
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
```

Expected result: repeated requests should identify both Web Server 1 and Web Server 2. The order may vary, and the first request is not guaranteed to go to VM1.

## Step 5: test fault tolerance

On VM1, stop Apache:

```
sudo systemctl stop apache2
```

On VM3, keep sending requests:

```
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
```

Expected result: only VM2 should respond after HAProxy health checks detect that VM1 is unavailable. Restore VM1:

```
sudo systemctl start apache2
sudo systemctl status apache2
```

Wait a few seconds and test again. VM1 should return to service after its health check succeeds.

# Practical 2 — School Fee Portal: Load Balancing and Fault Tolerance

Goal: Serve the same mock fee portal from VM1 and VM2. HAProxy distributes requests and redirects them when a backend fails. The page uses fake records only.

## Step 1: VM1 — install Apache and create the fee page

```
sudo apt update
sudo apt install apache2 curl -y
sudo systemctl enable --now apache2
hostname -I
sudo tee /var/www/html/index.html > /dev/null <<'EOF'
<!doctype html>
<html><head><title>School Fee Portal</title></head>
<body>
<h1>School Fee Payment Portal</h1>
<p>Backend: VM1</p>
<p>Student ID: DEMO001</p>
<p>Name: Sample Student</p>
<p>Total Fee: Rs. 25000</p>
<p>Paid: Rs. 15000</p>
<p>Balance: Rs. 10000</p>
</body></html>
EOF
curl http://localhost
```

## Step 2: VM2 — create the same mock records

```
sudo apt update
sudo apt install apache2 curl -y
sudo systemctl enable --now apache2
hostname -I
sudo tee /var/www/html/index.html > /dev/null <<'EOF'
<!doctype html>
<html><head><title>School Fee Portal</title></head>
<body>
<h1>School Fee Payment Portal</h1>
<p>Backend: VM2</p>
<p>Student ID: DEMO001</p>
<p>Name: Sample Student</p>
<p>Total Fee: Rs. 25000</p>
<p>Paid: Rs. 15000</p>
<p>Balance: Rs. 10000</p>
</body></html>
EOF
curl http://localhost
```

## Step 3: VM3 — configure HAProxy

Back up the file, open it, remove the old contents, and paste the complete configuration. Replace backend IPs with your actual addresses.

```
sudo apt update
sudo apt install haproxy curl -y
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.backup
sudo nano /etc/haproxy/haproxy.cfg
```

```
global
    log /dev/log local0
    log /dev/log local1 notice
    daemon
    maxconn 2048

defaults
    log global
    mode http
    option httplog
    timeout connect 5s
    timeout client 30s
    timeout server 30s

frontend fee_front
    bind *:80
    default_backend fee_servers

backend fee_servers
    balance roundrobin
    option httpchk GET /
    server vm1 192.168.1.101:80 check
    server vm2 192.168.1.102:80 check
```

Validate the configuration, start HAProxy, and record VM3 IP:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl enable --now haproxy
sudo systemctl restart haproxy
hostname -I
```

## Step 4: test and fail over

```
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
```

Open http://LOAD_BALANCER_IP in a browser on a computer that can reach VM3. The backend label may change between requests.

Stop VM1 Apache:

```
sudo systemctl stop apache2
```

On VM3 test again:

```
curl http://LOAD_BALANCER_IP
```

The page should still be served by VM2. Restore VM1:

```
sudo systemctl start apache2
```

**Important:** This is a static display-only portal. If a real fee payment or record update were implemented, the two web servers must use the same database or shared data service; otherwise their records could differ.

# Practical 3 — Physical Server Consolidation (Logical Simulation)

Goal: measure the resources assigned to VMs, prepare a consolidation plan, and compare before/after figures. Running several VMs on one PC demonstrates virtualization and logical consolidation; it does not mean physical servers were actually removed.

## Step 1: collect resource information on each VM

Run these commands on each VM and save the outputs or take screenshots:

```
hostname
nproc
free -h
df -h
uptime
top
```

For a short CPU sample, install sysstat and run mpstat:

```
sudo apt update
sudo apt install sysstat -y
mpstat 1 5
```

In top, press q to exit. Record the VM name, assigned CPU/RAM, actual CPU usage, memory used, disk usage, and the workload running on it.

## Step 2: create a before/after table

| Measure | Before | After (planned/observed) |
|---|---|---|
| **Number of physical hosts** | Enter actual count | Enter actual count; do not guess |
| **Number of VMs** | Enter actual count | Enter actual count |
| **CPU allocation** | Record vCPU totals | Record vCPU totals |
| **RAM allocation** | Record RAM totals | Record RAM totals |
| **CPU utilization** | Use mpstat/top | Use mpstat/top after change |
| **RAM usage** | Use free -h | Use free -h after change |
| **Workloads** | List each service | List its new VM/host |

## Step 3: make a consolidation plan

5. List all services and the CPU/RAM they require.
6. Group services that can safely share a VM or host. Keep enough free CPU and RAM for peak use.
7. Decide which workloads to move and where they will run.
8. Move or redeploy the workloads only if your lab allows it. Test each service after the move.
9. Repeat the resource commands and compare actual figures to the baseline.
10. Write down risks such as resource contention, a larger impact if the host fails, and insufficient capacity during peaks.

## Example plan (illustrative only)

The following is a planning example, not measured data. Replace it with your actual VM inventory.

| Before | Possible consolidated plan |
|---|---|
| **VM-A: web service, 1 vCPU / 1 GB RAM** | Combined host runs web service and log service |
| **VM-B: log service, 1 vCPU / 1 GB RAM** | Use a suitably sized VM/host with enough spare CPU and RAM |
| **VM-C: monitoring, 1 vCPU / 1 GB RAM** | Keep monitoring separate if isolation/reliability requires it |

# Practical 4 — School Exam Portal: Load Balancing

Goal: create a mock exam login page on two backends and send browser/HTTP requests through HAProxy. The form is only a display demo and does not authenticate users.

## Step 1: VM1 — create the exam page

```
sudo apt update
sudo apt install apache2 curl -y
sudo systemctl enable --now apache2
hostname -I
sudo tee /var/www/html/index.html > /dev/null <<'EOF'
<!doctype html>
<html><head><title>School Exam Portal</title></head>
<body>
<h1>School Examination Portal</h1>
<p>Backend server: VM1</p>
<form>
<label>Dummy username:</label><input type="text" name="username"><br><br>
<label>Dummy password:</label><input type="password" name="password"><br><br>
<button type="button">Login Demo</button>
</form>
</body></html>
EOF
curl http://localhost
```

## Step 2: VM2 — create the same page

```
sudo apt update
sudo apt install apache2 curl -y
sudo systemctl enable --now apache2
hostname -I
sudo tee /var/www/html/index.html > /dev/null <<'EOF'
<!doctype html>
<html><head><title>School Exam Portal</title></head>
<body>
<h1>School Examination Portal</h1>
<p>Backend server: VM2</p>
<form>
<label>Dummy username:</label><input type="text" name="username"><br><br>
<label>Dummy password:</label><input type="password" name="password"><br><br>
<button type="button">Login Demo</button>
</form>
</body></html>
EOF
curl http://localhost
```

## Step 3: VM3 — configure HAProxy

```
sudo apt update
sudo apt install haproxy curl -y
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.backup
sudo nano /etc/haproxy/haproxy.cfg
```

```
global
    log /dev/log local0
    log /dev/log local1 notice
    daemon
    maxconn 2048

defaults
    log global
    mode http
    option httplog
    timeout connect 5s
    timeout client 30s
    timeout server 30s

frontend exam_front
    bind *:80
    default_backend exam_servers

backend exam_servers
    balance roundrobin
    option httpchk GET /
    server vm1 192.168.1.101:80 check
    server vm2 192.168.1.102:80 check
```

Replace the example IPs, save and exit nano, then run:

Validate the configuration, start HAProxy, and record VM3 IP:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl enable --now haproxy
sudo systemctl restart haproxy
sudo systemctl status haproxy
hostname -I
```

## Step 4: test load balancing

```
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
```

Open http://LOAD_BALANCER_IP in a browser. Use dummy usernames only. The button does not log in or store data.

## Step 5: stop one backend

```
sudo systemctl stop apache2
```

Run the curl command again on VM3. It should receive the page from the remaining healthy server. Restore the stopped server with:

```
sudo systemctl start apache2
```

**Important:** In a real exam system, the login session must be shared or stored centrally, and exam answers must be saved in a shared database. Do not store real exam credentials in this demo.

# Practical 5 — Multiple VMs Communicating by Private IP

Goal: confirm that two VMs can reach each other and use a TCP client/server program. VM1 is the server; VM2 is the client.

## Step 1: find the IPs and test the network

On VM1 and VM2:

```
hostname -I
ip -br addr
```

From VM2, replace VM1_IP and run:

```
ping -c 4 VM1_IP
```

If ping fails, confirm both VMs use the same private virtual network. Some firewalls block ping even when TCP is allowed, so also test the actual application port after starting the server.

## Step 2: VM1 — create server.py

```
sudo apt update
sudo apt install python3 -y
nano server.py
```

Paste this Python code into nano:

```
import socket

HOST = "0.0.0.0"
PORT = 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen(1)

print("Server waiting for connection on port 5000...")
conn, addr = server.accept()
print("Connected by:", addr)

message = conn.recv(1024).decode()
print("Received from client:", message)
conn.sendall("Message received successfully".encode())

conn.close()
server.close()
```

Save with Ctrl+O, Enter, Ctrl+X. Start the server and leave the terminal open:

```
python3 server.py
```

## Step 3: allow the port if UFW is active

On VM1, check the firewall:

```
sudo ufw status
```

If it is active, allow port 5000 only from VM2:

```
sudo ufw allow from VM2_IP to any port 5000 proto tcp
```

## Step 4: VM2 — create client.py

```
sudo apt update
sudo apt install python3 -y
nano client.py
```

Paste this code, replacing VM1_IP with VM1's actual private IP address:

```
import socket

HOST = "VM1_IP"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

message = input("Enter message: ")
client.sendall(message.encode())

response = client.recv(1024).decode()
print("Server response:", response)
client.close()
```

Save, exit, and run the client on VM2:

```
python3 client.py
```

Type: Hello from VM2

## Expected output

```
Enter message: Hello from VM2
Server response: Message received successfully
```

On VM1, the server terminal should show the client address and the received message. This server accepts one client per run; restart it to test again.

# Practical 6 — Online Examination Portal: Load Balancing and Fault Tolerance

Goal: deploy two identical mock examination application pages, route requests through a separate load balancer, and verify that the page remains accessible if one backend stops.

## Step 1: VM1 — create the mock online exam page

```
sudo apt update
sudo apt install apache2 curl -y
sudo systemctl enable --now apache2
sudo tee /var/www/html/index.html > /dev/null <<'EOF'
<!doctype html>
<html><head><title>Online Examination Portal</title></head>
<body>
<h1>Online Examination Portal</h1>
<p>Application server: VM1</p>
<p>Candidate: DEMO001</p>
<p>Question: 2 + 2 = ?</p>
<form><input type="radio" name="q1"> 3
<input type="radio" name="q1"> 4
<input type="radio" name="q1"> 5
<button type="button">Submit Demo</button></form>
</body></html>
EOF
curl http://localhost
```

## Step 2: VM2 — deploy the same page

```
sudo apt update
sudo apt install apache2 curl -y
sudo systemctl enable --now apache2
sudo tee /var/www/html/index.html > /dev/null <<'EOF'
<!doctype html>
<html><head><title>Online Examination Portal</title></head>
<body>
<h1>Online Examination Portal</h1>
<p>Application server: VM2</p>
<p>Candidate: DEMO001</p>
<p>Question: 2 + 2 = ?</p>
<form><input type="radio" name="q1"> 3
<input type="radio" name="q1"> 4
<input type="radio" name="q1"> 5
<button type="button">Submit Demo</button></form>
</body></html>
EOF
curl http://localhost
```

## Step 3: VM3 — configure the load balancer

```
sudo apt update
sudo apt install haproxy curl -y
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.backup
sudo nano /etc/haproxy/haproxy.cfg
```

```
global
    log /dev/log local0
    log /dev/log local1 notice
    daemon
    maxconn 2048

defaults
    log global
    mode http
    option httplog
    timeout connect 5s
    timeout client 30s
    timeout server 30s

frontend online_exam_front
    bind *:80
    default_backend online_exam_back

backend online_exam_back
    balance roundrobin
    option httpchk GET /
    server vm1 192.168.1.101:80 check
    server vm2 192.168.1.102:80 check
```

Replace the example IPs with actual addresses; save and exit nano.

Validate the configuration, start HAProxy, and record VM3 IP:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl enable --now haproxy
sudo systemctl restart haproxy
sudo systemctl status haproxy
hostname -I
```

## Step 4: test normal operation

```
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
```

Open http://LOAD_BALANCER_IP in your browser. Backend identity should show which server answered. The sample form is not connected to a database and does not save answers.

## Step 5: test failover and recovery

On VM1:

```
sudo systemctl stop apache2
```

On VM3:

```
curl http://LOAD_BALANCER_IP
```

The response should come from VM2 after health-check detection. Restore VM1:

```
sudo systemctl start apache2
sudo systemctl status apache2
```

Result: continued page access was demonstrated using a healthy backend. A real exam portal additionally needs a shared database/session store, backups, and controls to prevent duplicate answer submissions.

# Practical 7 — Client-Server Communication Through a Load Balancer

Goal: VM1 and VM2 run simple Python HTTP servers on different ports. VM3 runs HAProxy and routes browser/HTTP requests to either backend.

## Step 1: VM1 — create backend directory and page

```
sudo apt update
sudo apt install python3 -y
mkdir -p ~/backend
cat > ~/backend/index.html <<'EOF'
<h1>Response from Backend Server 1 (VM1)</h1>
EOF
hostname -I
```

Start the Python server and leave this terminal open:

```
python3 -m http.server 8001 --bind 0.0.0.0 --directory ~/backend
```

## Step 2: VM2 — create the second backend

```
sudo apt update
sudo apt install python3 -y
mkdir -p ~/backend
cat > ~/backend/index.html <<'EOF'
<h1>Response from Backend Server 2 (VM2)</h1>
EOF
hostname -I
```

Start the server and leave this terminal open:

```
python3 -m http.server 8002 --bind 0.0.0.0 --directory ~/backend
```

## Step 3: allow backend ports if UFW is active

On VM1, if UFW is active:

```
sudo ufw status
sudo ufw allow from LOAD_BALANCER_IP to any port 8001 proto tcp
```

On VM2, if UFW is active:

```
sudo ufw status
sudo ufw allow from LOAD_BALANCER_IP to any port 8002 proto tcp
```

Before configuring HAProxy, test direct connectivity from VM3:

```
curl http://VM1_IP:8001
curl http://VM2_IP:8002
```

## Step 4: VM3 — configure HAProxy

```
sudo apt update
sudo apt install haproxy curl -y
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.backup
sudo nano /etc/haproxy/haproxy.cfg
```

```
global
    log /dev/log local0
    log /dev/log local1 notice
    daemon
    maxconn 2048

defaults
    log global
    mode http
    option httplog
    timeout connect 5s
    timeout client 30s
    timeout server 30s

frontend http_front
    bind *:80
    default_backend http_back

backend http_back
    balance roundrobin
    option httpchk GET /
    server vm1 192.168.1.101:8001 check
    server vm2 192.168.1.102:8002 check
```

Replace the example IP addresses with actual VM1 and VM2 addresses. Save and exit nano, then run:

Validate the configuration, start HAProxy, and record VM3 IP:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl enable --now haproxy
sudo systemctl restart haproxy
sudo systemctl status haproxy
hostname -I
```

## Step 5: test through the load balancer

```
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
curl http://LOAD_BALANCER_IP
```

Expected result: requests should show responses from both backends. Stop VM1's Python server with Ctrl+C, then repeat curl. HAProxy should route to VM2 after detecting the failed health check. Restart the Python server on VM1 to confirm recovery.

# Troubleshooting Checklist

## 1. The VMs cannot ping each other

- Confirm every VM is powered on.
- Confirm all VMs use the same private virtual network (VMware NAT on the same VMnet, or the same VirtualBox NAT Network/host-only network).
- Run hostname -I and verify you copied the correct IP.
- Run ip -br addr and confirm the adapter has an IPv4 address.
- Do not use 127.0.0.1 when connecting from another VM.

## 2. curl reports connection refused or times out

- On Apache backends, run sudo systemctl status apache2.
- On HAProxy, run sudo systemctl status haproxy and sudo haproxy -c -f /etc/haproxy/haproxy.cfg.
- Test each backend directly from VM3 before testing the load balancer: curl http://VM1_IP and curl http://VM2_IP.
- For the Python server in Practical 7, check that the server terminal is still running and use the exact port 8001 or 8002.
- If UFW is active, allow the required port from the private VM IP. Do not open ports to the public internet.

## 3. HAProxy does not start

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo journalctl -u haproxy --no-pager -n 50
```

Common causes are a misspelled directive, a missing indentation/parameter, an incorrect IP address format, or two frontends both binding to port 80. Use one complete configuration and check it before restarting.

## 4. Only one backend seems to respond

- Make sure each backend page clearly identifies VM1 or VM2.
- Use separate curl commands rather than reusing a browser page cache.
- Check health-check status and logs: sudo journalctl -u haproxy --no-pager -n 50.
- Confirm both backend pages return HTTP content directly from VM3.

## 5. How to stop and restart services

| Action | Apache backend | HAProxy load balancer |
|---|---|---|
| **Check status** | sudo systemctl status apache2 | sudo systemctl status haproxy |
| **Stop service** | sudo systemctl stop apache2 | sudo systemctl stop haproxy |
| **Start service** | sudo systemctl start apache2 | sudo systemctl start haproxy |
| **Restart service** | sudo systemctl restart apache2 | sudo systemctl restart haproxy |

# Final lab-record checklist

- Write down the VM names and actual private IP addresses.
- Include a simple topology: Client → VM3 (HAProxy) → VM1 / VM2.
- Take screenshots of commands and their actual outputs.
- Record results actually observed; do not paste expected output as if it were measured.
- For consolidation, say whether it was a logical simulation or a real workload migration.
- For fee/exam examples, use dummy data only. State that static HTML does not perform real authentication, payment processing, or exam submission.
- When finished, stop any Python servers with Ctrl+C. Keep the VMs for later practicals if you will reuse them.